

Sovereign AI-Powered IDS/SIEM

Consolidated Implementation Plan

Synthesized from the Gemini research report + Qwen technical plan, fact-checked against current (August 2026) licensing and hosting status, with a corrected build sequence.

1. Executive Summary

Both source documents converge on the same core idea: a layered, 100%-open-source Intrusion Detection & SIEM stack built from Zeek/Suricata (network), Wazuh/Sysmon (endpoint), Kafka/Flink (streaming), XGBoost+ONNX (tabular ML), MISP (threat intel), and Grafana/ClickHouse (visualization). That core is sound and still fully buildable at zero license cost.

Where the two reports needed correction:

- TheHive is no longer free. Both plans lean on it for incident case management. As of 2026 it's commercial-only (confirmed via the project's own GitHub notice — v3/v4 archived since 2023, v5+ is a paid product). This plan swaps in DFIR-IRIS, a genuinely free, actively maintained equivalent.
- Some “free forever” cloud claims have shrunk. Oracle's Always-Free ARM tier was cut in June 2026 from 4 vCPU/24GB to a hard 2 vCPU/12GB cap. This still matches the “lab tier” both reports assumed as a minimum — it just means there's no more free headroom above that.
- A couple of “free” tools are free-with-an-asterisk (Redpanda's BSL license, CrowdSec's capped free blocklist, Keep's AI-correlation feature). None of these block the build — they just need to be used within their real limits rather than treated as unconditionally free.
- The engineering corrections in the Qwen document are legitimate and kept as-is: in-process ONNX inference instead of per-flow Triton gRPC calls, tree ensembles (not transformers) on tabular flow data, and Suricata/ZeeK functional separation.

The rest of this document is the corrected, buildable plan.

2. Comparing the Two Source Reports

	Gemini report (research doc)	Qwen plan (technical doc)
Strength	Deep dataset coverage (UNSW-NB15, CIC-IDS2017/18, TON_IoT, NSL-KDD), tool-selection rationale, citations, licensing caveats (it even flagged the Triton CVEs itself)	Concrete engineering fixes: in-process ONNX vs. network round-trips, correct model family per data type, directory structure, latency/test benchmarks
Weakness	Treats “free” as static — several tools it names (TheHive, uncapped Oracle free tier) have since changed	Says relatively little about why specific datasets/algorithms were chosen, and inherits the same TheHive assumption
Roadmap style	Rollout order for the telemetry stack (Wazuh → Sysmon → Zeek/Suricata → OpenVAS → CrowdSec → deception/ML), ~6 weeks	Rollout order for the ML/streaming pipeline (env setup → features → Flink/ONNX → storage → SOC copilot → HITL), 6 phases

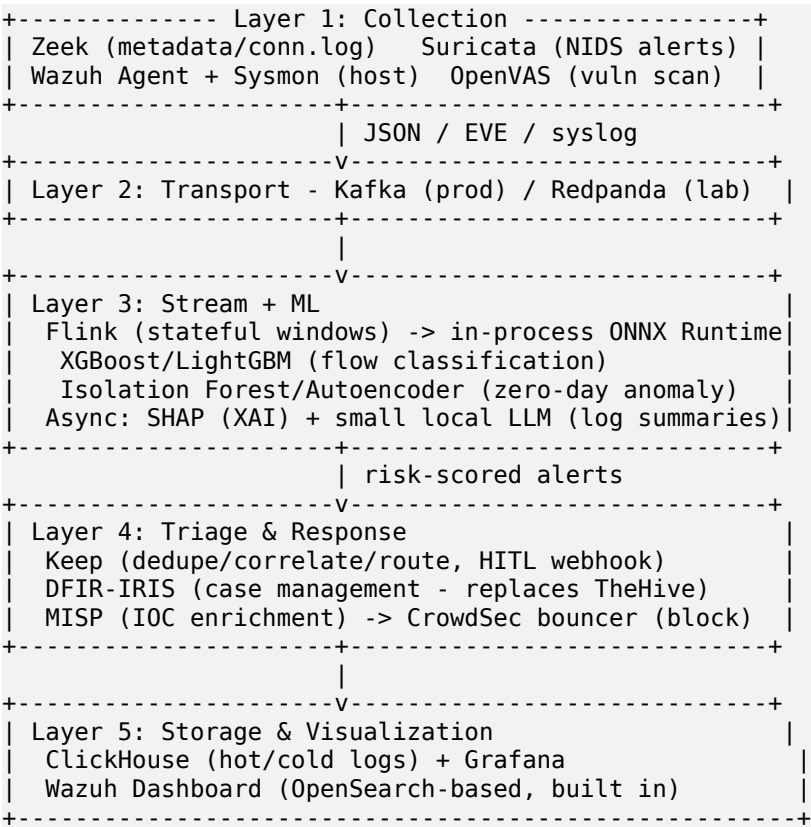
These two roadmaps aren't actually competing — one is “get telemetry flowing,” the other is “turn telemetry into AI-driven detection.” Section 5 merges them into a single sequence, because doing them out of order (e.g.

building the ML pipeline before Wazuh/Zeek/Suricata are reliably producing clean logs) means training on garbage data.

One more thing worth flagging that neither report mentions:

Published >98–99% accuracy numbers on UNSW-NB15/CIC-IDS2017 (which the Gemini report cites, and which the Qwen plan sets as a test target) are well known in NIDS research circles to be partly inflated by non-behavioral artifacts in those datasets (e.g. near-constant TTLs, timestamp/flow-ID leakage separating attack vs. benign traffic almost trivially). Treat a >98% validation score on these specific benchmarks as “the pipeline works,” not as “this will catch 98% of real attacks on your network.” Validate separately against your own captured traffic before trusting the numbers.

3. Corrected Reference Architecture



Note: modern Wazuh ships its own Wazuh Indexer, an Apache-2.0 OpenSearch fork — not raw Elasticsearch. Same RAM realities as Elasticsearch (~16GB comfortable minimum for the indexer alone at any real event volume), but no separate license concern.

4. Final Tool Stack (with license reality)

Layer	Tool	License / cost status
Network telemetry	Zeek, Suricata	Free, BSD/GPL, no change
Endpoint	Wazuh + Sysmon	Free, GPLv2, confirmed still fully free / no paywall

Layer	Tool	License / cost status
Vuln scanning	OpenVAS / Greenbone Community Edition	Free
Threat intel	MISP	Free, open source
Transport	Apache Kafka (prod) or Redpanda Community (lab)	Kafka: Apache 2.0. Redpanda: BSL — free for self-hosted internal use, can't resell as a hosted service
Stream processing	Apache Flink	Free, Apache 2.0
Model serving	ONNX Runtime (in-process); Ollama/vLLM only for the SOC-copilot LLM	Free
Case management	DFIR-IRIS (replaces TheHive)	Free, actively maintained, open source
Alert triage / AIOps	Keep (core workflow engine + HITL webhooks)	MIT / open source core; AI auto-correlation is Cloud/Enterprise-only — skip it, not needed
Active response	CrowdSec	Engine + community blocklist free (capped ~3k IPs unless you contribute signals)
Storage / Viz	ClickHouse, Grafana	Free, Apache 2.0
Hosting (optional free cloud)	Oracle Cloud Always Free (Ampere A1)	Free but capped at 2 OCPU / 12GB total (cut from 4/24 in June 2026)

5. Merged Phased Roadmap

Phase 0 — Decide your deployment tier first (½ day)

Pick one before building anything:

- Local lab (your own machine/VM, 16GB+ RAM recommended) — best for learning and iterating fast.
- Oracle free-tier cloud (2 OCPU/12GB Ampere A1) — remote, always-on, but tight; skip Flink/Kafka at this size and use Redpanda + a lightweight Python/ONNX consumer instead of full Flink.
- Hybrid: build/train locally, deploy only the lightweight detection agent to the free-tier VM.

Phase 1 — Foundational visibility (Week 1)

Deploy Wazuh manager + agents on all endpoints. Get basic log collection and the Wazuh dashboard working before adding anything else. This is your data source of truth for everything downstream — don't skip validating it.

Phase 2 — Enhanced endpoint telemetry (Week 2)

Add Sysmon on Windows endpoints (SwiftOnSecurity config as a starting ruleset), confirm Wazuh is ingesting Sysmon events.

Phase 3 — Network visibility (Weeks 3-4)

Deploy Zeek (metadata-only, no alerting) and Suricata (ET Open rules, alerting) on a tap/SPAN port. Integrate both into Wazuh via custom decoders. Confirm conn.log / dns.log / eve.json are flowing.

Phase 4 — Proactive scanning (Week 5)

Stand up OpenVAS/Greenbone, run scheduled scans, correlate findings against asset inventory.

Phase 5 — Streaming + ML pipeline (Weeks 6-8)

- Stand up Kafka (or Redpanda if resource-constrained) as the transport layer.
- Preprocess UNSW-NB15/CIC-IDS2017/TON_IoT (impute, encode, scale, SMOTE-Tomek for class imbalance).
- Train XGBoost/LightGBM + Isolation Forest, export to ONNX (skl2onnx).
- Load the ONNX model in-process inside a Flink job (or, at lab scale, a simple Python Kafka consumer calling onnxruntime) — this avoids the original per-flow Triton network round-trip bottleneck.
- Benchmark: replay PCAPs with tcpreplay, confirm sub-50ms scoring latency and non-null inference output.

Phase 6 — Triage, case management, response (Weeks 9-10)

- Deploy Keep, connect it to Wazuh/Flink alerts, build a workflow that enriches with MISP and routes high-risk alerts.
- Deploy DFIR-IRIS for case management (not TheHive).
- Connect CrowdSec bouncers, but gate every automated block behind Keep's HITL webhook approval — never auto-block without human sign-off, especially early on while your false-positive rate is unknown.

Phase 7 — Storage, dashboards, XAI (Week 11)

Wire ClickHouse + Grafana for metrics/health dashboards. Add SHAP explanations to high-risk alerts so you (or an analyst) can see why a flow was flagged before acting on it.

Phase 8 — Validation (Week 12, ongoing)

- Run nmap scans and hydra brute-force attempts against a test container; confirm Suricata alerts, Zeek logs the flow, the ONNX model flags it, Keep raises it, and (only after manual approval) CrowdSec blocks it.
- Re-test monthly as you tune rules — this is a live system, not a one-time build.

6. Directory Structure (adjusted)

```
ids-cybersecurity-system/
|-- docker-compose.yml
|-- config/
|   |-- suricata/  zeek/  wazuh/  kafka/  flink/  keep/
|-- ml_pipeline/
|   |-- datasets/  preprocess.py  train.py  export_onnx.py  models/
|-- flink_processor/      # or a simpler python_scorer/ at lab scale
|   |-- src/main/java/  pom.xml
|-- soc_copilot/
|   |-- app.py  prompt_templates.py
|-- dfir_iris/      # replaces the old thehive/ directory
|-- docker-compose.override.yml
```

7. Risks & Operating Notes

- Don't auto-block anything until you trust the model. Keep everything behind Keep's HITL approval for at least the first month of live traffic.

- Triton (if you use it at all, for the LLM/SOC-copilot layer only) has had real CVEs (remote code execution / DoS in older versions) — pin recent versions and patch on a schedule if you deploy it. Since the plan now only uses it (or Ollama/vLLM) for the async LLM layer rather than the hot path, exposure is much lower than the original per-flow design.
- Free tiers drift. Oracle just cut its ARM allocation; treat any “always free” cloud claim as something to re-verify at deployment time, not something to bank on long-term.
- Benchmark accuracy ≠ real-world accuracy on these public datasets — validate against your own traffic.
- Hardware reality check: the full stack (Wazuh + Kafka + Flink + ClickHouse + Keep + DFIR-IRIS + MISP) genuinely needs 16GB+ RAM to run comfortably; below that, drop Flink for a plain Python/ONNX consumer and use Redpanda instead of Kafka.

8. Immediate Next Steps

- Confirm your deployment tier (Phase 0) — this determines whether you run full Flink/Kafka or the lightweight substitute.
- Stand up Wazuh first and get it stable before touching anything else.
- When you get to Phase 6, deploy DFIR-IRIS instead of TheHive — same docker compose up style install, actively maintained, and actually free.